FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION: HOTELPRO

1. Introduction

Project Title: HotelPro – Complete Hotel Booking and Management System

Team Members:

- Sanjana Patidar – Frontend Development
- Backend Development – Node.js and Express.js
- Database Management – MongoDB

1.1 Project Overview

HotelPro is a web-based hotel booking and management application developed using the MERN stack. It allows users to browse hotel rooms, view prices and amenities, book rooms for selected dates, view booking history, and cancel eligible bookings. Administrators can manage rooms, users, and bookings through an admin dashboard.

1.2 Purpose

The main purpose of HotelPro is to simplify hotel booking and management. It provides an online platform for customers to book rooms conveniently and helps administrators manage hotel information and booking records efficiently.

1.3 Features

- User registration and login.
- JWT-based authentication.
- Room listings with images, prices, and amenities.
- Room booking with check-in and check-out dates.
- Booking history and status management.
- Cancellation of eligible bookings.
- Admin dashboard for managing rooms, users, and bookings.
- MongoDB database integration.
- Responsive and user-friendly interface.

2. Architecture

HotelPro follows the MERN stack architecture, consisting of MongoDB, Express.js, React.js, and Node.js.

2.1 Frontend

The frontend is developed using React.js and Vite. React components create the user interface for registration, login, room browsing, booking history, and the admin dashboard. The frontend communicates with the backend through HTTP requests to REST APIs.

2.2 Backend

The backend is developed using Node.js and Express.js. It handles authentication, room management, booking operations, and administrative functions. It validates requests, checks user permissions, and communicates with MongoDB using Mongoose.

2.3 Database

MongoDB stores application data in document-based collections. Mongoose is used to define schemas and perform database operations.

The main collections are:

- **Users:** Stores name, email, password hash, phone number, and user role.
- **Rooms:** Stores room type, description, price, images, and amenities.
- **Bookings:** Stores user ID, room ID, check-in date, check-out date, total price, contact details, and booking status.

The backend retrieves and updates database records according to user requests.

3. Setup Instructions

3.1 Prerequisites

The following tools are required:

- Node.js and npm
- MongoDB or MongoDB Atlas
- Visual Studio Code
- Git
- A modern web browser

3.2 Installation

Step 1: Open the HotelPro project folder in Visual Studio Code.

Step 2: Install backend dependencies by executing the following commands:

cd server

npm install

Step 3: Create a .env file inside the server directory using .env.example as a reference.

Example configuration:

PORT=5000

MONGODB_URI=mongodb://127.0.0.1:27017/hotelpro

CLIENT_URL=http://localhost:5173

For MongoDB Atlas, use the appropriate connection string. Configure any additional variables required by the project.

Step 4: Open another terminal and install frontend dependencies.

cd client

npm install

Step 5: Ensure MongoDB is running and the frontend API URL points to the backend server.

4. Folder Structure

HotelPro/

```
├── client/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
│
├── server/
│   ├── src/
│   │   ├── routes/
│   │   ├── models/
│   │   ├── middleware/
│   │   ├── server.js
│   │   └── seedAdmin.js
```

```
│   ├── package.json
│   └── .env.example
│
├── .gitignore
└── README.md
```

Note: Adjust the structure according to the actual project files.

4.1 Client

The client directory contains the React frontend. The public folder stores static assets, while src/components contains reusable components. The App.jsx file organizes the main interface, and main.jsx initializes the React application.

4.2 Server

The server directory contains the backend application. The routes directory defines API endpoints, models contain database schemas, and middleware handles authentication. The server.js file initializes Express and connects the application to MongoDB.

5. Running the Application

The frontend and backend must run simultaneously in separate terminals.

5.1 Backend

Execute the following commands:

cd server

npm run dev

The backend normally runs at http://localhost:5000. The terminal should confirm that MongoDB is connected and the API server has started.

5.2 Frontend

Execute the following commands in another terminal:

cd client

npm run dev

Open the local URL displayed by Vite, usually http://localhost:5173. If the port is occupied, Vite may select another port.

6. API Documentation

HotelPro uses REST APIs to exchange data between the frontend and backend. The following endpoints are representative; verify the exact paths and methods against the implemented backend routes.

6.1 Authentication APIs

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate a user
GET	/api/auth/me	Retrieve the current user, if implemented

Example registration request:

```
{
  "name": "Sanjana",
  "email": "sanjana@example.com",
  "password": "example-password",
  "phone": "9876543210"
}
```

The server validates the information and returns a response indicating whether registration was successful.

6.2 Room APIs

Method	Endpoint	Description
GET	/api/rooms	Retrieve room listings
GET	/api/rooms/:id	Retrieve individual room details, if implemented
POST	/api/rooms	Add a room; admin access required
PUT/PATCH	/api/rooms/:id	Update room details, if implemented
DELETE	/api/rooms/:id	Delete a room, if implemented

These APIs allow the application to display room information and support room management operations.

6.3 Booking APIs

Method	Endpoint	Description
POST	/api/bookings	Create a booking
GET	/api/bookings/mine	Retrieve the current user's bookings

PATCH/PUT /api/bookings/:id Update a booking, if implemented

The cancellation endpoint depends on the route implemented in the backend.

Example booking request:

```
{
  "roomId": "ROOM_ID",
  "checkIn": "2026-12-10",
  "checkOut": "2026-12-12"
}
```

Protected requests require a valid JWT in the authorization header:

Authorization: Bearer YOUR_JWT_TOKEN

The backend validates the room ID, booking dates, and authentication before processing a booking request. A successful response may contain the booking details and status.

7. Authentication and Authorization

HotelPro uses JSON Web Tokens (JWT) to authenticate users and protect restricted API endpoints.

7.1 Authentication Process

1. The user registers by providing the required details.
2. The backend validates the information and securely hashes the password.
3. During login, the backend verifies the user's credentials.
4. After successful login, a JWT is issued.
5. The frontend stores the authentication information according to the implementation.
6. The token is sent with protected API requests.
7. The backend verifies the token before granting access.

7.2 Authorization

The application supports customer and administrator roles.

- **Customer:** Can browse rooms, create bookings, view personal booking history, and cancel eligible bookings.
- **Administrator:** Can manage rooms, users, and bookings through the admin dashboard.

Authentication verifies a user's identity, while authorization determines which operations the user is allowed to perform.

8. User Interface

The HotelPro interface is designed to provide a simple and user-friendly experience. Its main screens include:

- **Home Page:** Displays the application introduction and available rooms.
- **Login and Registration:** Allows users to access or create accounts.
- **Room Listing:** Displays room images, prices, descriptions, and amenities.
- **User Dashboard:** Shows booking history and available booking actions.
- **Admin Dashboard:** Provides sections for managing rooms, users, and bookings.

Screenshots of these pages should be added to demonstrate the application's functionality.

9. Testing

Testing is performed to verify that the application works correctly and handles invalid requests appropriately.

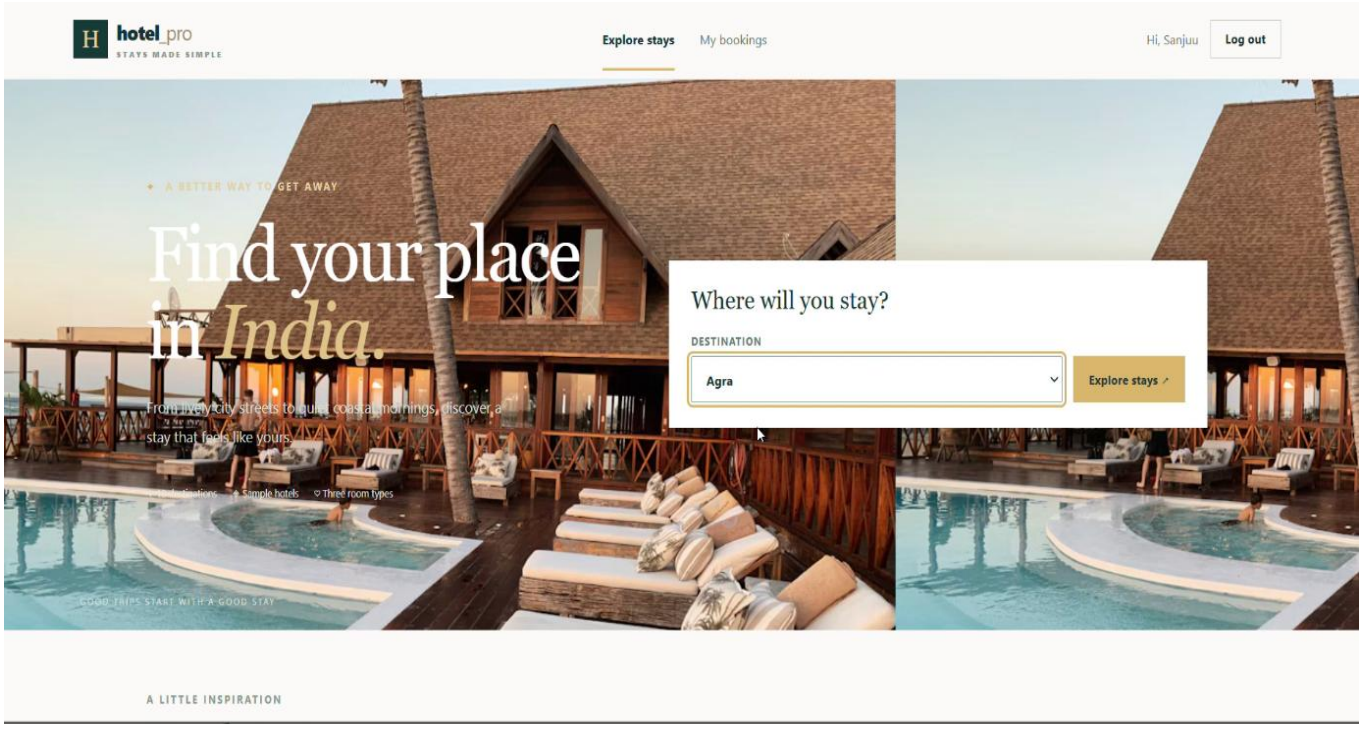
The following test cases are considered:

1. Verify registration and login with valid and invalid credentials.
2. Check that available rooms and their details are displayed correctly.
3. Verify booking creation with valid check-in and check-out dates.
4. Check that invalid dates are rejected.
5. Verify booking history and cancellation functionality.
6. Check that only administrators can access administrative operations.
7. Verify database connectivity and frontend-backend communication.

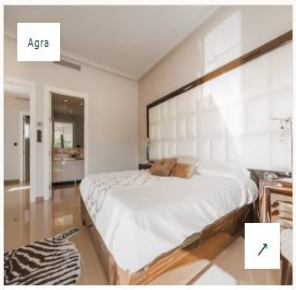
Browser developer tools, terminal logs, and an API testing tool such as Postman can be used for testing, where applicable.

10. Screenshots or Demo

Screenshots can be added to demonstrate the main features of HotelPro.



A LITTLE INSPIRATION

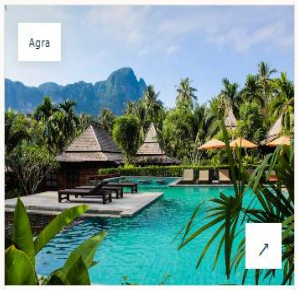


AGRA, INDIA ★ 4.3

Maple Agra Inn

Demo hotel in Agra with comfortable rooms for different budgets.

₹2,499 / night onwards View rooms →

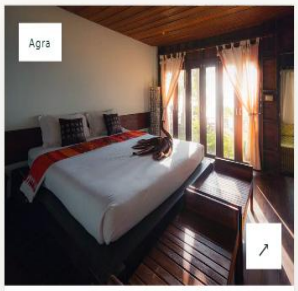


AGRA, INDIA ★ 4.3

Palm Grove Agra Inn

Demo hotel in Agra with comfortable rooms for different budgets.

₹2,499 / night onwards View rooms →

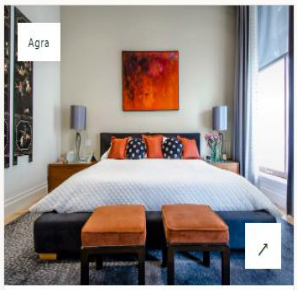


AGRA, INDIA ★ 4.3

Royal Agra Inn

Demo hotel in Agra with comfortable rooms for different budgets.

₹2,499 / night onwards View rooms →

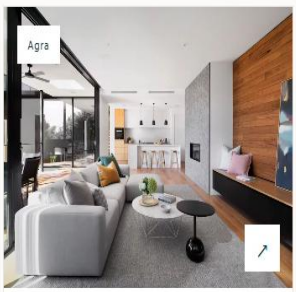


AGRA, INDIA ★ 4.3

Silver Agra Inn

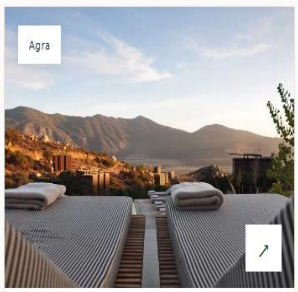
Demo hotel in Agra with comfortable rooms for different budgets.

₹2,499 / night onwards View rooms →



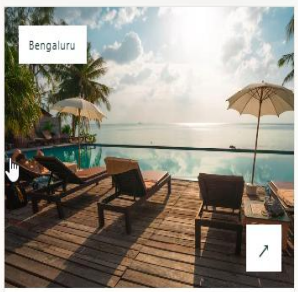
AGRA, INDIA ★ 4.3

Sunrise Agra Inn



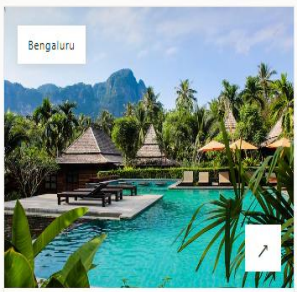
AGRA, INDIA ★ 4.3

Urban Nest Agra Inn



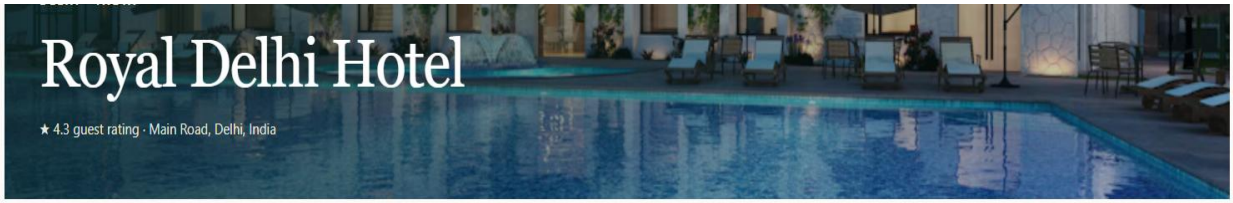
BENGALURU, INDIA ★ 4.3

Emerald Bengaluru Lodge



BENGALURU, INDIA ★ 4.3

Golden Bengaluru Lodge



Royal Delhi Hotel

★ 4.3 guest rating · Main Road, Delhi, India

ROOMS & RATES

Find your kind of comfort

Choose your room

Standard

Up to 2 guests

Wi-Fi · Air conditioning · TV

₹2,499 per night

Deluxe

Up to 2 guests

Wi-Fi · Air conditioning · TV · Breakfast

₹3,999 per night

Suite

Up to 4 guests

Wi-Fi · Air conditioning · TV · Breakfast · Living area

₹6,999 per night

YOUR STAY

Make it yours

Check-in

11-11-2028

Check-out

12-11-0028

Guests

2 guests

Book this room

YOUR TRAVEL PLANS

My bookings.

All your stays, gathered in one place.



BENGALURU

Confirmed

Silver Bengaluru Lodge

Deluxe · 2 guest(s)

2/13/2028 — 4/14/2028

₹2,43,939 ID: D9DE1338

Cancel booking



JAIPUR

Confirmed

Urban Nest Jaipur Retreat

Suite · 2 guest(s)

3/12/2029 — 3/15/2029

₹20,997 ID: D9DE132B

Cancel booking

Live Demo URL:
https://drive.google.com/file/d/117HJePZQy5yq08gUt9rx9mVnvRhOT_nr/view?usp=sharing

GitHub Repository URL: https://github.com/Sanjana-Patidar28/Hotel_Pro

11. Known Issues

Potential issues that should be checked before submission include:

- The application may not load data if the backend or database is unavailable.
- Incorrect environment variables can prevent database connectivity.
- Authentication errors may prevent protected operations.
- Invalid booking dates or unavailable rooms may cause booking requests to fail.
- Port conflicts may require the frontend or backend to use another port.

Only issues that apply to the actual submitted application should be reported as known issues.

12. Future Enhancements

The following improvements could be introduced in future versions:

1. Integration of an online payment gateway.
2. Email or SMS notifications for booking confirmations.
3. Room search, filtering, and sorting.
4. Customer reviews and ratings.
5. Improved booking availability checks and reports.
6. Automated testing and enhanced security.
7. Better deployment monitoring and error handling.

13. Conclusion

HotelPro is a full-stack web application developed using MongoDB, Express.js, React.js, and Node.js. It combines room browsing, user authentication, booking management, and administrative functionality in one platform. The project demonstrates practical knowledge of frontend development, REST APIs, database integration, and role-based access control. It provides an efficient and user-friendly solution for hotel booking and management.